SAHAYAK

Two-Person Project Structure & Implementation Plan

Person A — Frontend | Person B — Backend

1. Project goal

Build Sahayak as a mobile-friendly Progressive Web App (PWA) that helps people understand scholarship requirements, ask questions in Tamil or English, upload documents, create and submit an application through a configured workflow, track status, set reminders, and request human support. Begin with one scholarship and a clearly labelled demo submission flow. Do not imply that a demo submission reached a government portal.

2. Recommended technology stack

Layer	Technology	Responsibility / note
Frontend	React + TypeScript + Vite	Pages, components, routing, responsive PWA UI
Styling	CSS Modules + CSS variables	Consistent colours, spacing, typography, accessible states
Language UI	react-i18next	Reviewed Tamil and English interface text; add languages after testing
Backend API	Python + FastAPI + Pydantic	REST endpoints, validation, service orchestration
Database	PostgreSQL + SQLAlchemy + Alembic	Durable records and schema migrations
Knowledge retrieval	pgvector + multilingual-e5-base	Search approved scholarship passages
AI answer generation	One evaluated LLM API (e.g., Claude API)	Answer only from retrieved evidence; return source references
Speech	Bhashini APIs or another tested provider	Tamil speech-to-text and text-to-speech; validate quality early
PDF/OCR	pdfplumber + Tesseract (Tamil/English data)	Extract text; present extracted fields as suggestions, not verified facts
Authentication	Firebase Authentication	Frontend sign-in; backend must verify tokens and authorize each request
Private file storage	Amazon S3 or equivalent private object storage	Certificates, uploaded documents, temporary audio; never public by default
Background jobs	Celery + Redis + Celery Beat	Long-running OCR, processing, and scheduled reminders
Testing	pytest + Playwright	API tests and end-to-end user journeys
Local development	Docker Compose	Repeatable local services
Version control	Git + GitHub	Branches, pull requests, issue tracking, shared documentation

3. Repository structure

Create one repository named sahayak. Person A owns frontend/; Person B owns backend/. Both review shared/api/, docs/, evaluation/, and the end-to-end journey.

```
sahayak/
├── README.md
├── compose.yaml
├── .env.example
├── .gitignore
├── docs/
│   ├── product-scope.md
│   ├── user-journeys.md
│   ├── setup-guide.md
│   ├── api-contract.md
│   ├── security-and-privacy.md
│   └── demo-guide.md
├── shared/
│   └── api/
│       ├── openapi.yaml
│       ├── examples/
│       │   ├── chat-request.json
│       │   ├── chat-response.json
│       │   └── application-response.json
│       └── error-format.md
├── frontend/                          # PERSON A
│   ├── package.json
│   ├── vite.config.ts
│   ├── public/
│   │   ├── icons/
│   │   └── manifest.webmanifest
│   └── src/
│       ├── app/
│       │   ├── App.tsx
│       │   ├── router.tsx
│       │   └── providers.tsx
│       ├── pages/
│       │   ├── HomePage.tsx
│       │   ├── ChatPage.tsx
│       │   ├── LoginPage.tsx
│       │   ├── ApplicationsPage.tsx
│       │   ├── ApplicationFormPage.tsx
│       │   ├── ApplicationReviewPage.tsx
│       │   ├── ApplicationStatusPage.tsx
│       │   ├── DocumentsPage.tsx
│       │   ├── RemindersPage.tsx
│       │   ├── SupportPage.tsx
│       │   ├── GrievancesPage.tsx
│       │   └── admin/
│       │       ├── AdminDashboardPage.tsx
│       │       ├── KnowledgePage.tsx
│       │       ├── ApplicationReviewPage.tsx
│       │       └── SupportInboxPage.tsx
│       ├── components/
│       │   ├── layout/
│       │   ├── chat/
│       │   ├── forms/
│       │   ├── documents/
│       │   ├── status/
│       │   ├── feedback/
│       │   └── common/
│       ├── api/
│       │   ├── client.ts
│       │   ├── auth.api.ts
│       │   ├── chat.api.ts
│       │   ├── applications.api.ts
│       │   ├── documents.api.ts
│       │   ├── reminders.api.ts
│       │   └── support.api.ts
```

```
│       ├── hooks/
│       ├── types/
│       ├── locales/
│       │   ├── en/translation.json
│       │   └── ta/translation.json
│       ├── styles/
│       │   ├── tokens.css
│       │   └── global.css
│       └── tests/
├── backend/                          # PERSON B
│   ├── pyproject.toml
│   ├── Dockerfile
│   ├── alembic.ini
│   ├── migrations/
│   ├── app/
│   │   ├── main.py
│   │   ├── api/
│   │   │   ├── deps.py
│   │   │   └── v1/
│   │   │       ├── router.py
│   │   │       ├── health.py
│   │   │       ├── auth.py
│   │   │       ├── chat.py
│   │   │       ├── voice.py
│   │   │       ├── knowledge.py
│   │   │       ├── forms.py
│   │   │       ├── documents.py
│   │   │       ├── applications.py
│   │   │       ├── status.py
│   │   │       ├── reminders.py
│   │   │       ├── support.py
│   │   │       ├── grievances.py
│   │   │       └── admin.py
│   │   ├── core/
│   │   │   ├── config.py
│   │   │   ├── security.py
│   │   │   ├── permissions.py
│   │   │   ├── logging.py
│   │   │   └── errors.py
│   │   ├── db/
│   │   │   ├── session.py
│   │   │   └── base.py
│   │   ├── models/
│   │   ├── schemas/
│   │   ├── services/
│   │   │   ├── knowledge_ingestion.py
│   │   │   ├── retrieval.py
│   │   │   ├── chat.py
│   │   │   ├── speech.py
│   │   │   ├── document_extraction.py
│   │   │   ├── applications.py
│   │   │   ├── reminders.py
│   │   │   ├── notifications.py
│   │   │   ├── support.py
│   │   │   └── grievances.py
│   │   ├── adapters/
│   │   │   ├── llm_client.py
│   │   │   ├── bhashini_client.py
│   │   │   ├── firebase_auth.py
│   │   │   ├── object_storage.py
│   │   │   ├── scholarship_demo.py
│   │   │   └── notification_provider.py
│   │   ├── workers/
│   │   │   ├── celery_app.py
```

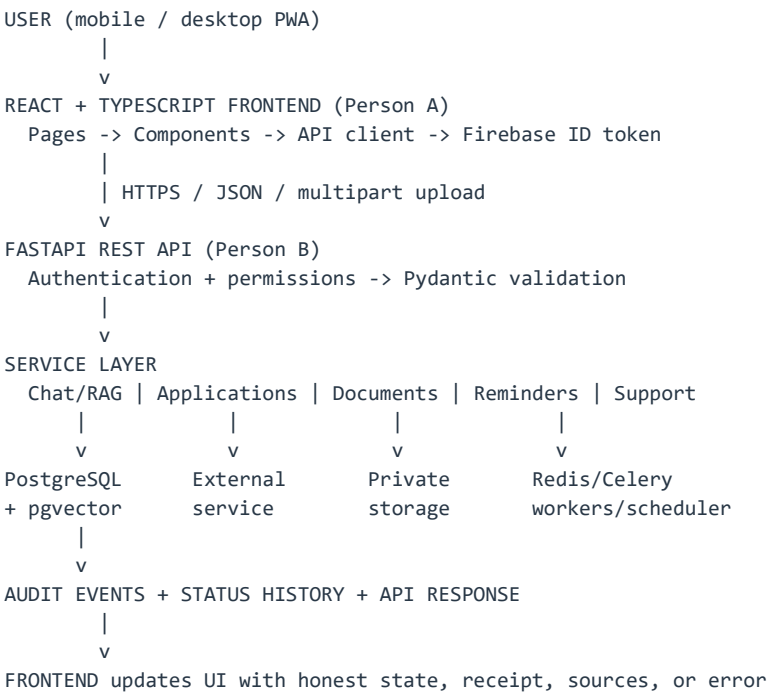
```
│   │   │   ├── document_tasks.py
│   │   │   └── reminder_tasks.py
│   │   └── utils/
│   └── tests/
│       ├── unit/
│       ├── integration/
│       └── fixtures/
├── evaluation/
│   ├── scholarship_questions.csv
│   ├── expected_sources.csv
│   └── tamil-language-checklist.md
├── tests/
│   └── e2e/
│       ├── guest-chat.spec.ts
│       ├── application-journey.spec.ts
│       └── staff-review.spec.ts
└── infra/
    ├── docker/
    └── deployment-notes.md
```

4. Folder ownership and responsibilities

Area	Person A — Frontend	Person B — Backend
Project foundation	Vite app, routes, layout, shared components	FastAPI app, config, database connection, health endpoint
Authentication	Login/logout UI, session state, protected routes	Verify Firebase tokens; enforce ownership and staff roles
Chat + knowledge	Chat UI, language toggle, loading/error states, source citations	Ingest approved circulars, retrieve passages, generate evidence-grounded answers
Applications	Form steps, validation messages, draft/review/submit screens	Create/edit drafts, validate, submit via demo/authorised adapter, persist receipt and status
Documents	Upload, preview, progress, editable extracted-field UI	Validate file, private storage, OCR/extraction jobs, processing status
Voice	Mic permission, start/stop, transcript correction, audio playback	Speech provider integration and audio/text processing
Tracking/reminders	Status timeline and reminder screens	Status events, scheduler, notification delivery records
Support/grievances	Ticket forms, conversation UI, progress display	Ticket APIs, assignment, staff replies, audit trail
Admin	Knowledge upload/publish and application/support screens	Protected admin endpoints, approval/publish actions, role checks
Testing/release	Playwright journeys, accessibility, responsive/PWA checks	pytest, migrations, Docker services, monitoring, backups

5. System building flow

A request should travel through clear layers. Do not put business rules or secrets inside frontend components.



6. Core user journeys

A. Cited scholarship chat

- Guest opens Sahayak and chooses Tamil or English.
- User asks a question by typing; voice can be added after text chat works.
- Frontend sends POST /api/v1/chat with message and language.
- Backend searches only approved scholarship knowledge, sends relevant passages to the model, and returns answer plus source title/page.
- Frontend displays the answer and source links. If evidence is missing, show a safe fallback instead of inventing a rule.

B. Application and tracking

- User signs in and creates a draft.
- Frontend collects required details and uploads supporting documents.
- Backend validates and stores draft data; extraction suggestions remain editable.
- User reviews destination, fields, and attachments, then explicitly confirms submission.
- Backend records the actual adapter result and returns a reference number; demo submissions must be labelled DEMO.
- Tracking page reads persisted status and status history from the backend.

C. Document assistance

- Authenticated user uploads a permitted PDF or image.
- Backend checks size/type, stores the file privately, and creates a processing job.
- Worker extracts PDF text or runs OCR, then returns suggested fields with confidence/status where available.
- User corrects or confirms the values before they are copied into an application draft.
- Never call an extracted document authentic or officially verified unless an authorised verification process actually occurred.

7. API contract to agree on before coding

Feature	Endpoint	Purpose
Health	GET /api/v1/health	Check backend availability
Current user	GET /api/v1/me	Return current authenticated profile and permitted role
Chat	POST /api/v1/chat	Question, language; returns answer and sources
Knowledge	POST /api/v1/admin/knowledge	Staff-only upload/review workflow for approved source documents
Services	GET /api/v1/services	List available scholarship/service options
Applications	POST /api/v1/applications	Create a draft
Applications	GET /api/v1/applications	List only the current user's applications
Application detail	GET /api/v1/applications/{id}	Read an owned application
Draft edit	PATCH /api/v1/applications/{id}	Update a draft
Submit	POST /api/v1/applications/{id}/submit	Explicitly submit once through configured adapter
Status	GET /api/v1/status/{application_id}	Read current status and event history
Document upload	POST /api/v1/forms	Extract fields from uploaded document; not application submission
Job status	GET /api/v1/jobs/{id}	Poll long-running processing progress
Reminders	/api/v1/reminders	Create/list/update/cancel reminders
Support	/api/v1/support	Create tickets and exchange replies
Grievances	/api/v1/grievances	Create and track complaints
Staff review	/api/v1/admin/applications	Staff-only queue and permitted status updates

Use a consistent JSON error format, for example: {"error":{"code":"VALIDATION_ERROR","message":"Please check the highlighted fields.","details":[]}}. Document the exact request and response schemas in shared/api/openapi.yaml.

8. Minimum database entities

Entity	Key purpose
users, institutions, memberships	Identity, institution, and role membership
consents	Consent and notification preferences
services, workflow_versions	Service requirements and workflow configuration
knowledge_documents, knowledge_chunks	Approved circulars, metadata, chunks, embeddings, source pages
conversations, messages, message_sources	Chat history and citations
documents, document_extractions	Private file metadata, extraction output, user corrections
applications, application_events	Draft/application fields, receipt, status history
reminders, notification_deliveries	Due dates and delivery attempts
support_tickets, support_messages	Human help and staff responses
grievances, grievance_events	Complaint progress
jobs, audit_events	Background processing and security-sensitive actions

Every private record must have an owner (and institution where applicable). Enforce access checks in the backend, not only by hiding frontend buttons.

9. Build in milestones

Milestone	Person A delivers	Person B delivers	Definition of done
1. Foundation	App shell, navigation, API client, responsive layout	FastAPI, DB connection, migration, health route	Frontend successfully calls backend
2. Auth	Sign-in/out UI and session handling	Token verification and record-level permissions	Users cannot read another user's records
3. Cited chat	Chat UI, source panel, loading/failure states	Knowledge ingestion, retrieval, LLM response and citations	Tamil/English test questions return supported answers
4. Application MVP	Form, save draft, review, submit and receipt screens	Draft APIs, validation, demo submission adapter, status	One application can be saved, submitted once, and tracked

		events	after refresh
5. Document extraction	Upload/preview/progress/edit fields	Private storage, OCR, background job and job status	User can correct extracted fields and save them
6. Voice	Recording controls, transcript review, playback	Speech-to-text/text-to-speech provider	User can correct transcript before sending
7. Reminders/support	Reminder and support UI	Scheduler, notification records, ticket workflow	Saved reminder and staff reply persist
8. Admin and release	Admin screens, PWA, accessibility and E2E tests	Admin permissions, deployment, backups, monitoring and API tests	Complete end-to-end journey works on a phone and deployed environment

10. User-friendly requirements for every screen

- Keep primary navigation simple: Home, Ask Sahayak, My Applications; place secondary functions under a clear menu.
- Use plain language and one clear question per form step. Explain unfamiliar scholarship terms.
- Show loading, empty, error, permission-denied, offline, and service-unavailable states.
- Keep extracted values editable and show the original document alongside them where practical.
- Separate Save Draft, Review, and Submit. Ask for confirmation before submission.
- Do not claim success until the backend confirms it. If a request times out after submission starts, check status before retrying to avoid duplicates.
- Preserve drafts across refreshes and language changes; do not store sensitive documents in public caches.
- Make voice optional and always provide typing and transcript correction.
- Use accessible labels, keyboard focus, sufficient contrast, large touch targets, and readable Tamil fonts.
- Display sources for scholarship answers and clearly say when the approved knowledge base does not contain an answer.
- Show only features that are working; mark any demo-only workflow honestly.

11. Local setup and team workflow

- Create the repository and add the folder structure above.
- Add .env.example with variable names only; never commit real API keys, passwords, service-account files, or production data.
- Person B starts FastAPI, PostgreSQL, pgvector, Redis, and worker services through Docker Compose.
- Person A starts the Vite frontend and configures VITE_API_BASE_URL to point at the local backend.
- Keep endpoint schemas in shared/api/openapi.yaml. Person A can use mocked responses while Person B implements an endpoint.
- Use feature branches and pull requests. Avoid both people editing the same files without agreeing first.
- For each feature, agree on method/path, request, response, error codes, auth requirement, and example before implementation.
- Run backend tests and frontend/E2E tests before merging. Use synthetic documents and test accounts.
- Document how to run migrations, seed demo data, start services, and reset the demo safely.

12. MVP scope: what to build first

Do not build all future modules at once. The first complete vertical slice should be:

- Tamil/English language selection and a usable mobile layout.
- One scholarship circular reviewed by a human and published to the knowledge base.
- A cited chat answer with a clear fallback when evidence is missing.
- Login and one saved scholarship application draft.
- Review and explicitly confirmed demo submission with a clearly labelled demo reference.

- Persistent application status and a basic staff status-update screen.
- Basic access-control tests and a rehearsed demo.

After this slice is stable, add document autofill, voice, reminders, support tickets, grievances, and additional institutions/languages.

13. Acceptance checklist

- The frontend calls the backend using the agreed API contract.
- A Tamil question returns a relevant answer with a source and page/reference.
- Unsupported scholarship claims are not invented.
- Private documents are stored privately and are not included in the public knowledge base.
- Users can only access their own applications and files; staff routes require staff permission.
- A draft survives page refresh and server restart.
- Submission is explicit, duplicate-safe, and returns an honest acknowledgement.
- Status changes are stored as events and visible after refresh.
- OCR results are editable and are not labelled as officially verified.
- The app has loading, empty, error, offline, and permission-denied states.
- The main journey is tested on a real phone-sized viewport.
- Backup/restore and demo reset steps are documented.

14. Important implementation cautions

- The actual scholarship rules and current circulars must be supplied and reviewed; the planning documents alone are not enough to answer eligibility questions reliably.
- Real government-portal submission requires an authorised integration. Until then, use a clearly labelled demo adapter.
- Test Tamil OCR and speech recognition on representative examples before promising high accuracy.
- Keep secrets on the backend. Frontend environment variables are visible to users after build.
- Restrict uploads by file type and size, scan where feasible, use short-lived access links, and apply retention/deletion rules.
- Do not claim a document or application is verified unless the relevant authority or verified integration confirms it.

Build one complete, trustworthy journey first. Expand only after it works end to end.